



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Title: Web Architecture Analysis and XSS Vulnerability Identification

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course: Web Application Security Essentials

Course Code: BVWS102

Instructor: AHMED BUKAR

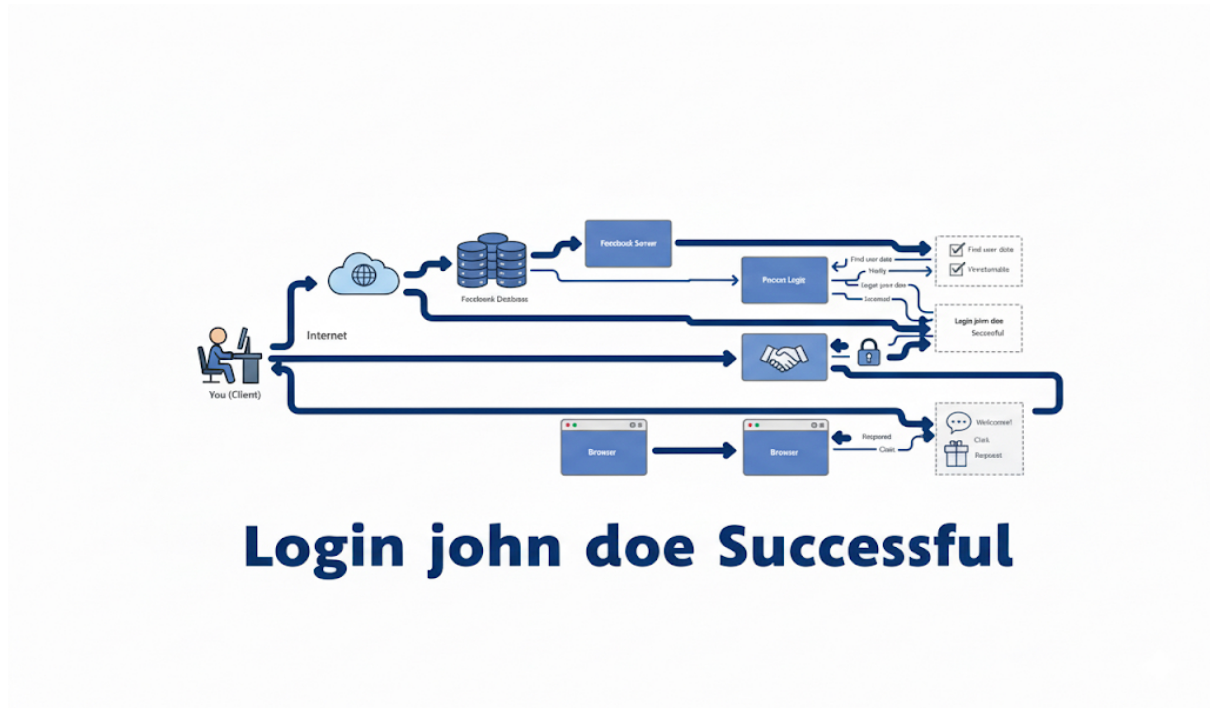
9/NOV/2025

EXERCISE 1

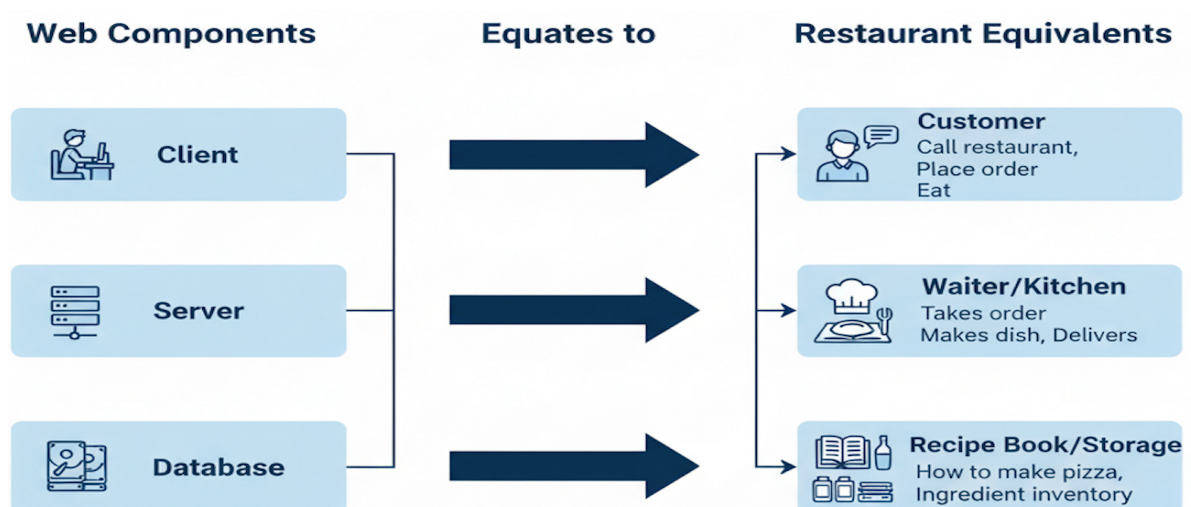
Three Tier Architecture



Facebook Login Scenario



Restaurant Analogy Mapping



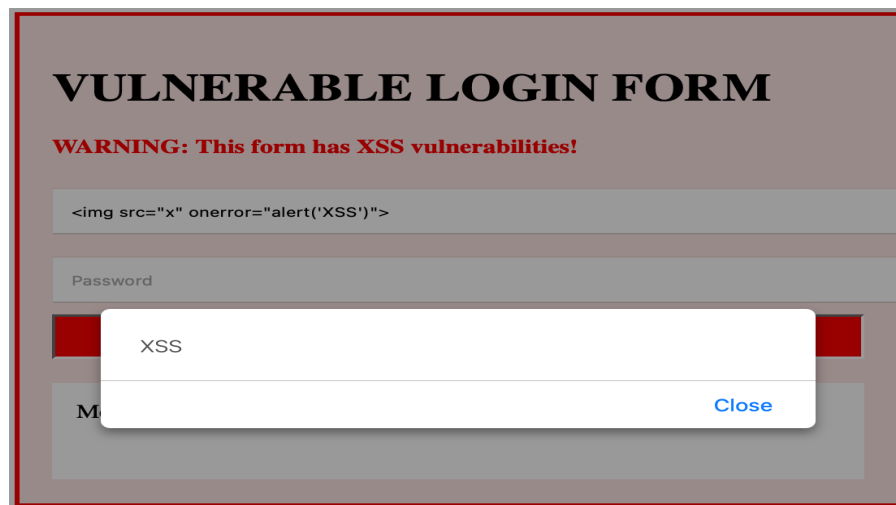
It is very dangerous if a customer can directly access the “recipe book” because

- It creates critical security vulnerabilities by exposing sensitive proprietary data and financial information, and it risks data integrity because the customer could maliciously or accidentally corrupt or delete essential "recipes" (data) without passing through the controlled validation steps of the "waiter/kitchen" (server).
- It creates critical security vulnerabilities by exposing sensitive proprietary data and financial information, and it risks data integrity because the customer could maliciously or accidentally corrupt or delete essential "recipes" (data) without passing through the controlled validation steps of the "waiter/kitchen" (server).

EXERCISE 2

Create a vulnerable vulnerable login form and try these additional XSS payloads in the username fields.

1. ``



The code `` is a concise Cross-Site Scripting (XSS) payload that relies on an HTML error to execute JavaScript. By setting the image source (`src="x"`) to an invalid path, the browser triggers the `onerror` attribute, which runs the included command, `alert('XSS')`. This action immediately confirms that the web application failed to neutralize the attacker's input, making it vulnerable to code injection.

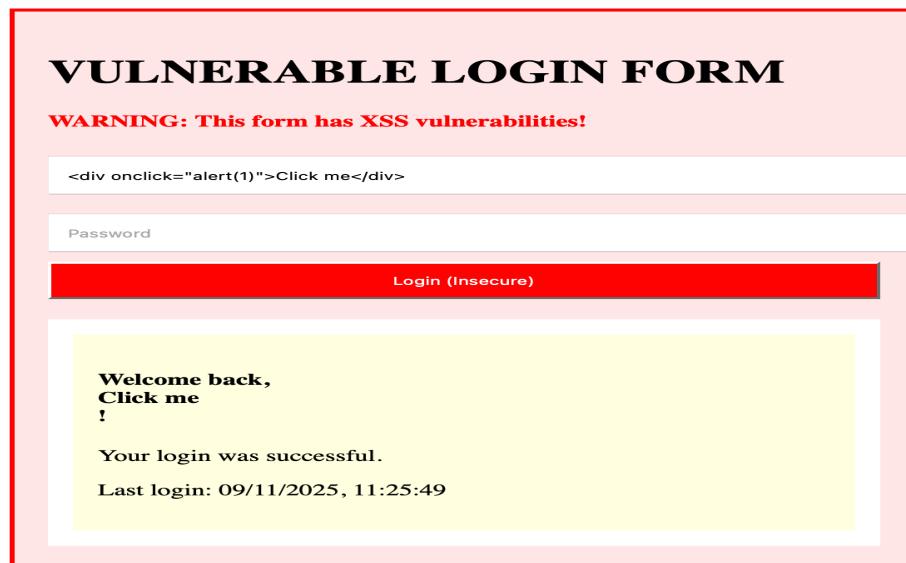
2. `<svg onload="alert('ICDFA')">`



The screenshot shows a web application titled "VULNERABLE LOGIN FORM" with a warning: "WARNING: This form has XSS vulnerabilities!". Below the warning, the payload `<svg onload="alert('ICDFA')">` is entered into the input field. The "Password" field is empty. The "Login (Insecure)" button is highlighted in red. Below the button, a yellow box displays the message: "Welcome back, Your login was successful. Last login: 09/11/2025, 11:24:42".

The code `<svg onload="alert('ICDFA')">` is a very common Cross-Site Scripting (XSS) payload that executes JavaScript when the SVG element successfully loads. Since the onload attribute is immediately triggered when the browser renders the `<svg>` tag, it runs the command `alert('ICDFA')` confirming the application's failure to sanitize input and demonstrating vulnerability to code injection.

3. `<div onclick="alert(1)">Click me</div>`



The screenshot shows the same "VULNERABLE LOGIN FORM" with the warning. The payload `<div onclick="alert(1)">Click me</div>` is entered into the input field. The "Password" field is empty. The "Login (Insecure)" button is highlighted in red. Below the button, a yellow box displays the message: "Welcome back, Click me ! Your login was successful. Last login: 09/11/2025, 11:25:49".

The code `<div onclick="alert(1)">Click me</div>` is an HTML element that executes the JavaScript command `alert(1)` only when a user interacts by clicking the created "Click me" area, proving the application is vulnerable to code injection via user-interaction event handlers like `onclick`.

EXERCISE 3

What specific characters does the ‘escape HTML’ function neutralize?

The escape HTML function neutralizes five special characters: &, <, >, ", and '. These characters are dangerous because browsers interpret them as part of HTML or JavaScript. By converting them into safe HTML entities

- < becomes <
- > becomes >
- " becomes "
- ' becomes '

By doing this, the browser no longer interprets them as code. Instead, they are displayed as plain text. For example, <script> is shown on the page as the word “<script>” instead of being executed as a real script

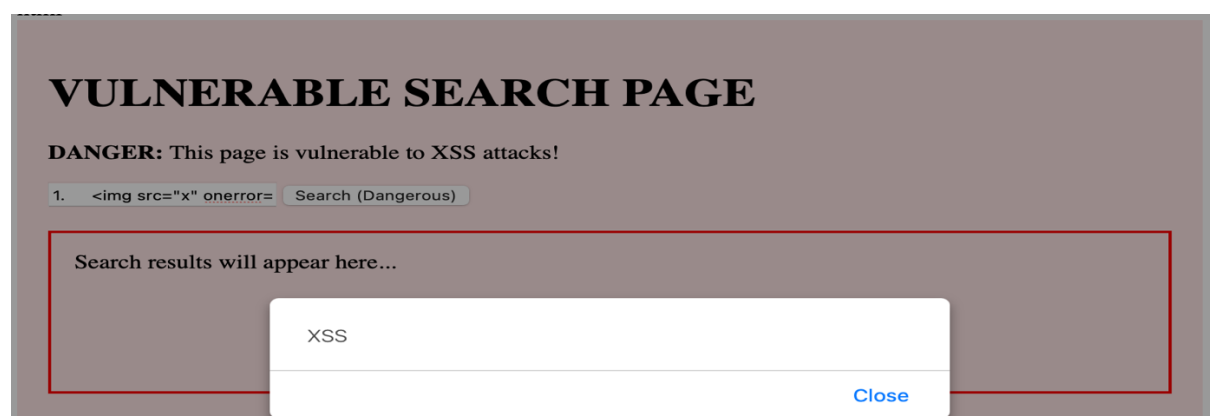
Why is displaying `<script>alert('XSS')</script>` as text safe?

Displaying <script>alert('XSS')</script> as text is safe because the browser no longer interprets it as executable code. The escaping process converts the < and > characters into harmless entities (< and >). As a result, the browser shows the string <script>alert('XSS')</script> on the page like normal text, instead of running it as a script. This prevents the malicious code from executing and keeps the application secure.

EXERCISE 4

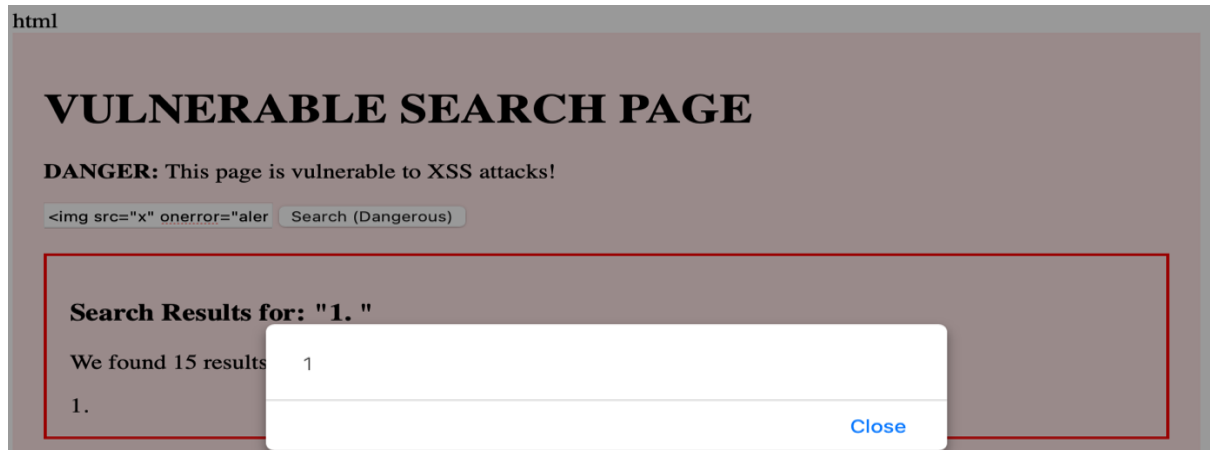
VULNERABLE SEARCH PAGE

<script>alert('XSS')</script>

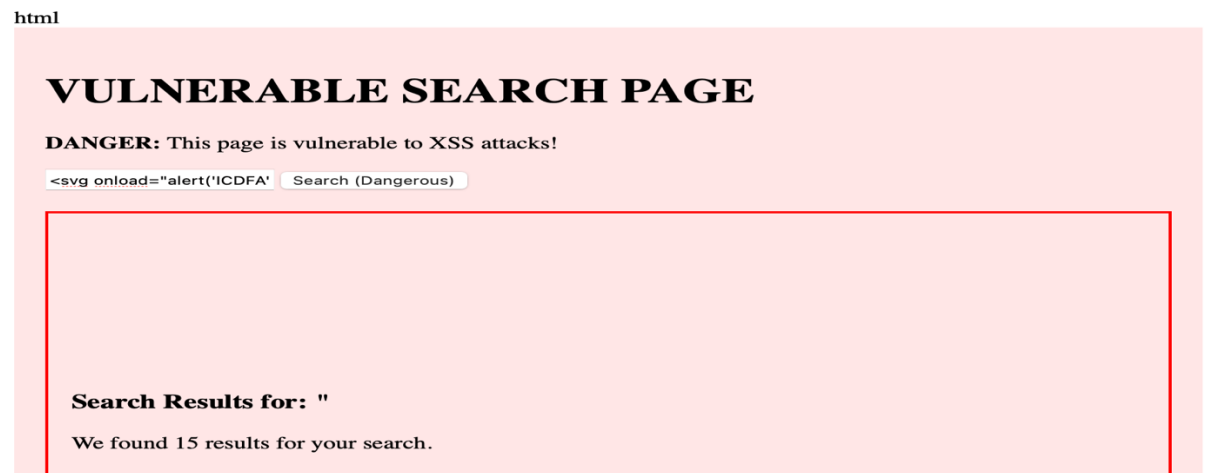


```

```



```
<svg onload="alert('ICDFA')">
```



SECURE SEARCH PAGE

```
<script>alert('XSS')</script>
```



```

```

html

SECURE SEARCH PAGE

SAFE: This page is protected against XSS attacks!

"**

We found 15 results for your search.


```
<svg onload="alert('ICDFA')">
```

html

SECURE SEARCH PAGE

SAFE: This page is protected against XSS attacks!

<svg onload="alert('ICDFA'

Search (Safe)

Search Results for: "<svg onload="alert('ICDFA')">"

We found 15 results for your search.

<svg onload="alert('ICDFA')">

PAYLOAD	VULNERABLE RESULT (vulnerable-search.html)	SECURE RESULT (secure-serach.html)
<script>alert('XSS')</script>	1.The browser executes the script immediately. 2.An alert box pops up with the message “XSS”.	1.The input is escaped. The page shows the text <script>alert('XSS')</script> instead of running it. 2.No alert appears.
	1.The image fails to load, triggering the onerror event.	1.The input is escaped. 2.The page displays the string as plain text. 3.No alert is triggered.

	2.An alert box pops up with the number 1 .	
<svg onload="alert('ICDFA') ">	1.The SVG element loads and immediately executes the onload event. 2. An alert box pops up with the text "ICDFA" .	1.The input is escaped. 2.The page shows the string <svg onload="alert('ICDFA')"> as harmless text. 3.No alert appears.

LAB ANALYSIS

PART A: WEB ARCHITECTURE

1. Explain why the database should never directly communicate with the client.

The database stores sensitive information (like usernames, passwords, and personal data), If the client (browser) talks directly to the database, attackers could send harmful commands (like SQL injection) and steal or change data. The server acts as a “middleman” that checks requests before they reach the database.

The database should never directly communicate with the client because it would expose sensitive data and allow attackers to run harmful commands. The server must control all access.

2. What security risks exist if client-side code contains hidden admin panels?

Client-side code runs in the browser, and attackers can view it by checking “View Source” or developer tools, If admin panels are hidden in the code, attackers can reveal them and try to access admin functions without permission.

The admin panels in client-side code are risky because attackers can uncover them and gain unauthorized access to admin features.

3. How does the three-tier architecture help with security?

The three-tier architecture improves security by separating responsibilities, ensuring the client cannot directly access the database, and allowing the server to enforce rules.

By separating them, you prevent direct access to sensitive data and reduce the chance of a single point of failure.

PART B: XSS VULNERABILITIES

1. What is the fundamental difference between how vulnerable and secure code handles user input?

Vulnerable code displays user input directly, while secure code sanitizes input to prevent execution of malicious scripts.

Vulnerable code: trusts user input and displays it directly (e.g., using `innerHTML`).

Secure code: sanitizes or escapes user input before displaying it, so dangerous characters are shown as text instead of running as code.

2. Why is `innerHTML` dangerous while `textContent` is safe?

Inner HTML interprets input as HTML, so `<script>` tags or event handlers can run (inner HTMLs dangerous because it executes code inside user input) while text content treats input as plain text, so `<script>` is shown as words, not executed (text Contents safe because it only displays text).

3. List three real-world consequences of XSS vulnerabilities.

- Hackers can steal cookies and impersonate users.
- Hackers can redirect users to fake websites (phishing).
- Hackers can deface websites or inject malicious ads.

PART C: SECURE DEVELOPMENT

1. Besides input sanitization, what other measures can prevent XSS attacks?

- Use Content Security Policy (CSP) to block unauthorized scripts.
- Avoid inline JavaScript and event handlers.
- Encode output properly depending on context (HTML, attributes, URLs).
- Avoiding inline scripts
- Proper output encoding are additional measures to prevent XSS.

2. Why should validation happen on both client and server sides?

Client-side validation gives quick feedback (e.g., “password too short”).

Server-side validation ensures data is truly safe before storing or processing.

Hackers can bypass client checks, so server checks are essential.

Validation should happen on both sides because client checks improve user experience, but server checks enforce real security.

3. What's the risk of relying only on client-side security measures?

Attackers can disable or bypass client-side checks using developer tools or direct API calls, without server checks, harmful data can enter the system and compromise security.

Relying only on client-side security is risky because attackers can bypass it, leaving the application vulnerable.